

GamutX

Dictionary

Python Basics

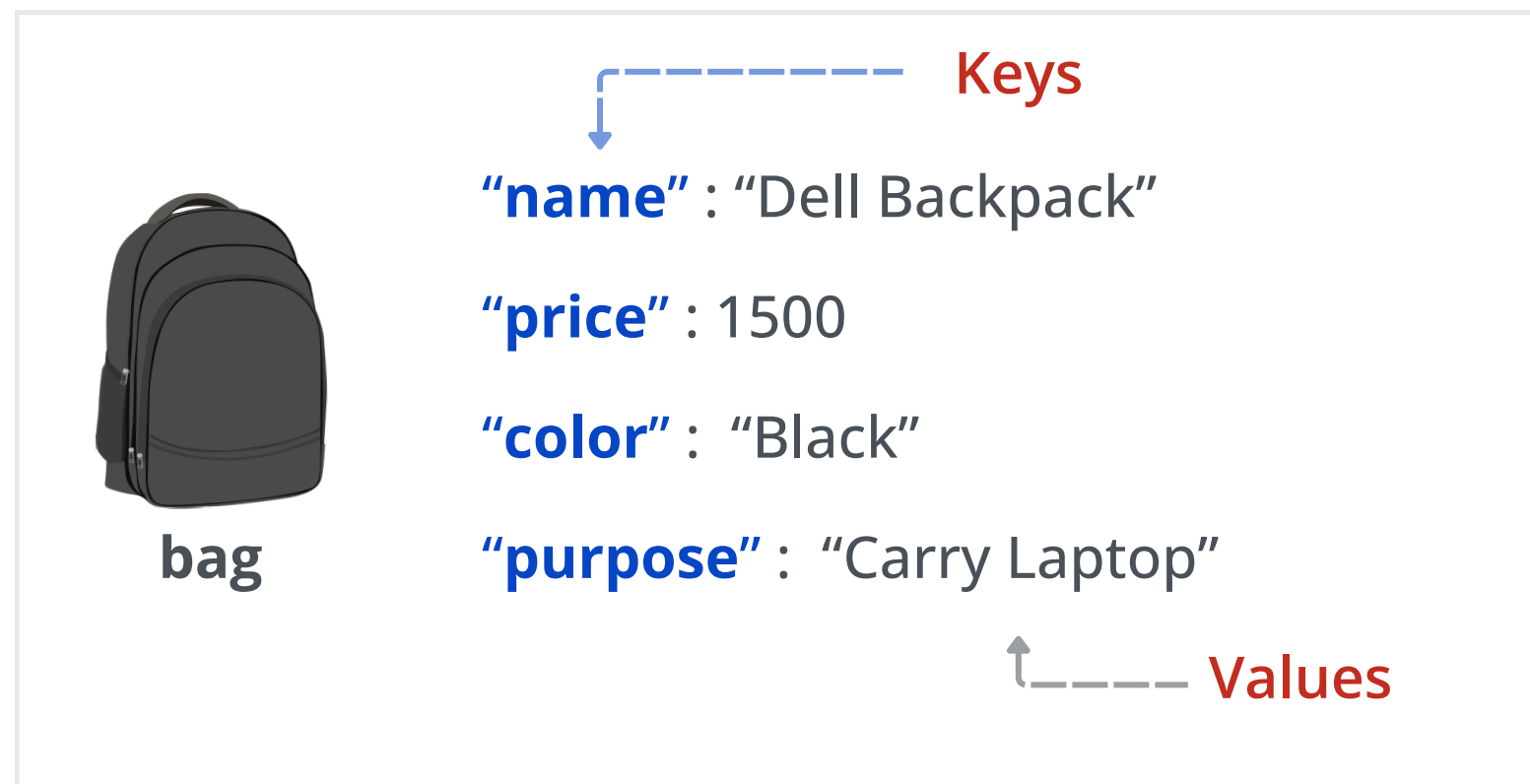


Introduction to Dictionary

- What is a Dictionary?
- Creating a Dictionary
- Ways of Creating Dictionaries
- Types of Data Stored in a Dictionary

What is a Dictionary?

- A dictionary in Python is a **collection of data stored in key-value pairs**
- A Dictionary is **represented using curly braces {} to enclose key-value pairs**



Key.

- The key acts like a label

Value

- The value holds the actual data

Creating a Dictionary

Syntax

```
dictionary_name = {key1: value1, key2: value2, ...}
```

Dictionary name

Each Key is separated by colon

Key value pair are enclose in curly braces

Note: Each Key-value pair are separated by comma

Let's see an example

```
student_info = {  
    "name": "John",  
    "age": 20,  
    "grade": "A"  
}  
  
print(student_info)  
  
# Output: {'name': 'John', 'age': 20, 'grade': 'A'}
```

name, age, grade → **Key**
"john", 20, "A" → **Value**

Ways of Creating Dictionaries

- we can create dictionaries in two main ways:
 - Using curly braces {}
 - Using the dict() constructor

Using Curly Braces {}

- This is the most common way we define a dictionary is **using curly braces {}**

Empty Dictionary

```
empty_dict = {}  
print(empty_dict)
```

```
# Output: {}
```

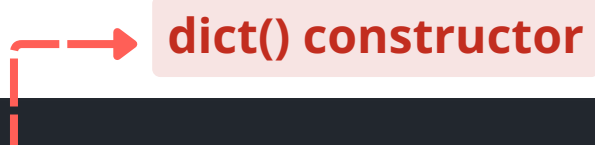
Return empty Dictionary

Creating a Dictionary with Key-Value Pairs

```
person = {  
    "name": "John",  
    "age": 30,  
    "city": "New York"  
}  
  
print(person)  
# {'name': 'John', 'age': 30, 'city': 'New York'}
```

Using the dict() Constructor

- Python also lets us create dictionaries using the built-in dict() Constructor.
- This is especially useful when we want to pass the key-value pairs directly as arguments.



```
person = dict( name="Alice", age=25, city="London" )  
print(person)  
  
# Output: {'name': 'Alice', 'age': 25, 'city': 'London'}
```

Inside the parentheses, we're passing key-value pairs in the format: **key=value**

Dictionary with Student Information

```
student = dict(name="John", grade="A", subject="Math", age=18)  
print(student)  
  
# {'name': 'John', 'grade': 'A', 'subject': 'Math', 'age': 18}
```

Dictionary with Product Details

```
product = dict(name="Laptop", brand="Dell", price=1200.50, stock=10)  
print(product)  
  
# {'name': 'Laptop', 'brand': 'Dell', 'price': 1200.50, 'stock': 10}
```

Types of Data Stored in a Dictionary

- In Python, dictionary **values** can hold **any data type** — from **simple numbers** to **complex objects**.
- Below is a **structured overview**, similar to what you shared for **JSON**.

1. String

- Text enclosed in quotes.
- Often used for names, labels, file paths.

```
shot = {  
    "name": "shot_010",  
    "status": "approved"  
}
```

String

2. Number

- Can be integer or float (decimal).
- Used for frame numbers, versioning, measurements, etc.

```
timing = {  
    "start_frame": 1001,  
    "duration": 128.5  
}
```

Number

3. Boolean

- Represents True or False.
- Useful for flags, conditions, toggles.

```
flags = {  
    "is_rendered": True,  
    "has_fx": False  
}
```

Boolean

4. None

- Represents an empty or unknown value.
- Often used as a placeholder.

```
review = {  
    "artist_notes": None,  
    "approved_by_supervisor": None  
}
```

None

5. List (Array)

- A collection of values in square brackets [].
- Can mix data types or hold repeating elements.

```
frame_data = {  
    "key_frames": [1001, 1050, 1100],  
    "camera_names": ["camMain", "camSide"]  
}
```

→ List

6. Tuple

- An immutable sequence (fixed once created).
- Common for fixed-length data like resolution.

```
output = {  
    "resolution": (1920, 1080),  
    "aspect_ratio": (16, 9)  
}
```

→ Tuple

7. Nested Dictionary

- A dictionary within a dictionary.
- Useful for grouping metadata or department-specific details.

```
asset = {  
    "name": "Alien",  
    "departments": {  
        "modeling": "done",  
        "texturing": "in_progress",  
        "rigging": "not_started"  
    }  
}
```

→ Nested Dictionary

Accessing Dictionary Elements

- Accessing Values - Using Keys
- Get All Keys - `keys()`
- Get All Values - `values()`
- Get All Key-Value Pairs - `items()`

Accessing Values - Using Keys

- The most common way to get a value from a dictionary is by using its key inside square brackets [].

Basic Usage

```
person = {"name": "Alice", "age": 25, "city": "London"}  
  
print(person["name"]) # Output: Alice  
print(person["age"])  # Output: 25
```

⚠ If we try to access a key which doesn't exist

```
print(person["address"])
```

Python will raise a **KeyError**.

```
ERROR!  
Traceback (most recent call last):  
  File "<main.py>", line 3, in <module>  
KeyError: 'address'
```

To avoid errors, use the **.get() method**

Concept Practice Task

Using Variables as Keys to Access Values

```
employee = {"name": "Sarah", "position": "Manager", "salary": 70000}
key = "salary"
print(employee[key]) # Output: 70000
```

- We can also use variables to store keys

Accessing Values in a Dictionary with Non-String Keys

```
numbers_dict = {1: "one", 2: "two", 3: "three"}

print(numbers_dict[1]) # Output: one
print(numbers_dict[2]) # Output: two
```

- Keys in a dictionary don't have to be strings — they can **also be integers, tuples, or any immutable type.**

Concept Practice Task

Accessing Values from a Product Dictionary

```
product = {"name": "Laptop", "brand": "Dell", "price": 800}

print(product["name"]) # Output: Laptop
print(product["brand"]) # Output: Dell
print(product["price"]) # Output: 800
```

Accessing Values in a Dictionary with Mixed Data Types

```
person = {"name": "John", "age": 30, "is_employed": True}

print(person["name"]) # Output: John
print(person["age"]) # Output: 30
print(person["is_employed"]) # Output: True
```

Key Takeaways

- If the key exists, you get the corresponding value.
- If the key doesn't exist, Python raises a `KeyError`.
- To avoid errors, use the `.get()` method: which doesn't raise an error if a key is missing.

Getting All Keys – .keys()

If we want to get all the keys in a dictionary, we can use the .keys() method.

```
person = {"name": "Alice", "age": 25, "city": "London"}  
keys = person.keys()  
print(keys) # dict_keys(['name', 'age', 'city'])
```

- This method **returns a view object** that displays a list of all the keys in the dictionary.
- The view object is dynamic, meaning if you change the dictionary, the view reflects these changes.

Getting All Values – .values()

Basic Dictionary

```
person = {"name": "Alice", "age": 25, "city": "London"}  
  
values = person.values()  
print(values) # dict_values(['Alice', 25, 'London'])
```

Empty Dictionary

```
empty_dict = {}  
empty_values = empty_dict.values()  
  
print(empty_values) # Output: dict_values([]) (Empty)
```

Employee Details

```
employee = {"name": "Bob", "position": "Developer", "salary": 70000}  
  
employee_values = employee.values()  
print(employee_values) # dict_values(['Bob', 'Developer', 70000])
```

Mixed Data Types

String integer Boolean

```
person_info = {"name": "Charlie", "age": 30, "is_student": False}  
  
info_values = person_info.values()  
print(info_values) # dict_values(['Charlie', 30, False])
```

Get All Key-Value Pairs - items()

- The items() method returns all key-value pairs from a dictionary.
- Each pair is returned as a tuple.
- The result is a special object called a view — which updates if the dictionary changes.

```
book = {"title": "1984", "author": "George Orwell", "year": 1949}
book_items = book.items()

print(book_items)
# Output:
# dict_items([('title', '1984'), ('author', 'George Orwell'), ('year', 1949)])
```

"author": "George Orwell"



('author', 'George Orwell') ← **tuple**

How it Work

- When we call **items()** on a dictionary:
- It returns a view object: **dict_items([...])**
- That view object holds a list of tuples, Each tuple contains:
 - The key
 - The value that goes with it

✓ This is useful when we need to display the entire dictionary — all at once.

Concept Practice Task

Empty Dictionary

```
empty_dict = {}  
  
empty_items = empty_dict.items()  
  
print(empty_items)  
  
# Output: dict_items([]) (Empty)
```

Basic Dictionary

```
car = {"brand": "Toyota", "model": "Camry", "year": 2021}  
car_items = car.items()  
  
print(car_items)  
  
# Output: dict_items([('brand', 'Toyota'), ('model', 'Camry'), ('year', 2021)])
```

Modifying Dictionaries

- Modifying Dictionaries
- Adding Key-Value Pairs
- Updating Values
- Updating Key-Value Pairs - update()
- Removing Key-Value Pairs - del Keyword
- Removing Key-Value Pairs - pop()

Modifying Dictionaries

Dictionaries are mutable, which means we can **change them after they're created**.

✓ What We Can Do:

- Add new key-value pairs
- Update existing values
- Update multiple pairs at once using the update() method
- Removing Key-Value Pairs – del Keyword
- Remove and retrieve a value using the pop() method

Adding a New Key-Value Pair

We can add new key-value pairs to a dictionary by assigning a value to a new key.

Basic Usage

```
person = {"name": "Alice", "age": 25}

person["city"] = "London"
print(person)
# Output:
# {'name': 'Alice', 'age': 25, 'city': 'London'}
```

Adding Multiple Key-Value Pairs

```
car = { "brand": "Toyota", "model": "Camry" }

car["year"] = 2021
car["color"] = "Blue"

print(car)
# Output: {'brand': 'Toyota', 'model': 'Camry', 'year': 2021, 'color': 'Blue'}
```

Using a Variable as the Key

```
inventory = { "apples": 10, "bananas": 20 }

new_fruit = "oranges"
inventory[new_fruit] = 15

print(inventory)
# Output: {'apples': 10, 'bananas': 20, 'oranges': 15}
```

Adding a Key-Value Pair to an Empty Dictionary

```
student = {}

student["name"] = "John"
student["grade"] = "A"

print(student)
# Output: {'name': 'John', 'grade': 'A'}
```

Updating a Existing Value

We can update the value associated with a specific key by simply assigning a new value to that key

Basic Usage

```
person = {"name": "Alice", "age": 25, "city": "London"}

person["age"] = 26

print(person)
# Output: {'name': 'Alice', 'age': 26, 'city': 'London'}
```

Updating Multiple Values

```
car = {"brand": "Toyota", "model": "Camry", "year": 2020 }

car["year"] = 2021
car["model"] = "Corolla"

print(car)
# Output:
# {'brand': 'Toyota', 'model': 'Corolla', 'year': 2021}
```

Updating a Value Using a Variable

```
inventory = { "apples": 10, "bananas": 20 }

new_apples = 15
inventory["apples"] = new_apples

print(inventory)
# Output: {'apples': 15, 'bananas': 20}
```

Updating Key-Value Pairs - update()

The update() method allows you to update multiple key-value pairs at once

Basic Usage

```
person = {"name": "Alice", "age": 25}

person.update({"age": 26, "city": "London"})

print(person)
# {'name': 'Alice', 'age': 26, 'city': 'London'}
```

Adding Multiple Key-Value Pairs

```
car = {
    "brand": "Toyota",
    "model": "Camry"
}

car.update(year=2021, color="Blue")
print(car)
# {'brand': 'Toyota', 'model': 'Camry', 'year': 2021, 'color': 'Blue'}
```

Updating a Dict with Another Dict

```
person = { "name": "Alice", "age": 25 }

new_info = { "age": 26, "city": "New York" }

person.update(new_info)

print(person)
# Output:
# {'name': 'Alice', 'age': 26, 'city': 'New York'}
```

New Key-Value Pairs to an Empty Dictionary

```
employee = {}

employee.update(name="Emma", position="Manager", salary=5000)
print(employee)
# {'name': 'Emma', 'position': 'Manager', 'salary': 5000}
```

Removing Key-Value Pairs - del Keyword

To remove a **specific item from the dictionary**, we can use the **del keyword**.

Basic Example

```
person = {"name": "Alice", "age": 25, "city": "London"}  
  
del person["city"]  
  
print(person) # Output: {'name': 'Alice', 'age': 25}
```

Deleting a Key-Value Pair from a Dictionary

```
person = {  
    "name": "Alice",  
    "age": 25,  
    "city": "New York"  
}  
  
del person["age"]  
  
print(person)  
# Output: {'name': 'Alice', 'city': 'New York'}
```

Deleting an Entire Dictionary

```
employee = {  
    "name": "Emma",  
    "position": "Manager",  
    "salary": 5000  
}  
  
del employee  
  
print(employee)
```

```
⊗ PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/Users/HP/Desktop  
Traceback (most recent call last):  
  File "c:\Users\HP\Desktop\tt\test.py", line 10, in <module>  
    print(employee) # NameError: name 'employee' is not defined  
NameError: name 'employee' is not defined  
○ PS C:\Users\HP\Desktop\tt>
```

Removing Key-Value Pairs - pop()

The **pop() method** allows us to **remove a key-value pair** and also **return the value**

Basic Usage

```
person = {"name": "Alice", "age": 25, "city": "London"}

age = person.pop("age")

print(age)      # 25
print(person)   # {'name': 'Alice', 'city': 'London'}
```

Using pop() with a Default Value

```
car = { "brand": "Toyota", "model": "Camry", "year": 2021}
removed_value = car.pop("color", "Key not found")
print(car)
# Output: {'brand': 'Toyota', 'model': 'Camry', 'year': 2021}

print("Removed value:", removed_value)
# Output: Removed value: Key not found
```

Removing the Last Key-Value Pair in a Dict

```
inventory = { "apples": 10, "bananas": 20, "oranges": 15 }

removed_value = inventory.pop("oranges")

print(inventory)
# Output: {'apples': 10, 'bananas': 20}

print("Removed value:", removed_value)
# Output: Removed value: 15
```

pop from an Empty Dictionary

```
empty_dict = {}

removed_value = empty_dict.pop("key", "No such key")

print(empty_dict)
# Output: {}

print("Removed value:", removed_value)
# Output: Removed value: No such key
```

Summary

- Let's say we have the following dictionary: →
 -  **person = {"name": "Alice", "age": 25, "city": "London"}**
- Now, here's how we can modify it:

Add a New Key-Value Pair	→ Set using square brackets	→ Ex. person["gender"] = "Female"
Update an Existing Value	→ Reassign using square brackets	→ Ex. person["age"] = 26
Update Multiple Values	→ Use update() method	→ Ex. person.update({"city": "Paris", "age": 27})
Remove a Dict	→ Use del keyword	→ Ex. del person["city"]
Remove a Key	→ Use pop() method	→ Ex. person.pop("age")

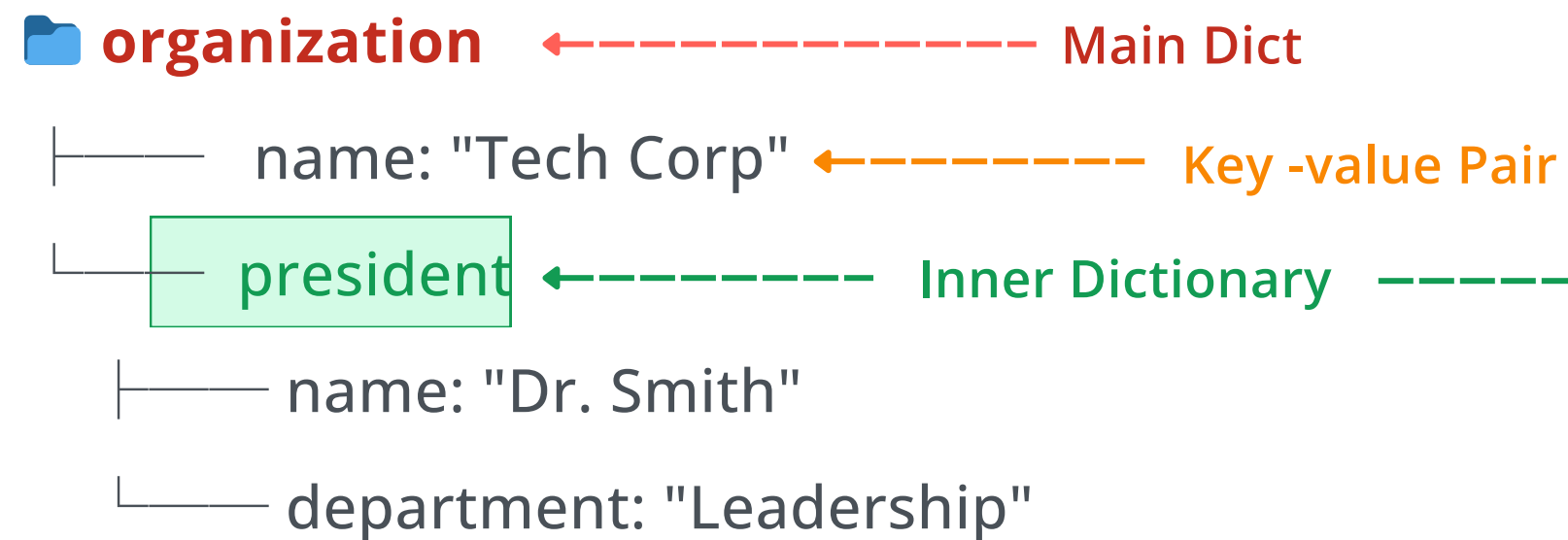
Nested Dictionaries

- What Is a Nested Dictionary?
- Creating Nested Dictionaries
- Accessing Nested Elements
- Modifying Nested Dictionaries

What Is a Nested Dictionary?

- A nested dictionary is a **dictionary inside another dictionary**.
- It helps you **store related data together** in a neat, organized way.
- It means that each **key in the outer dictionary holds another dictionary as its value**.

Analogy



Code example

```
organization = {  
    "name": "Tech Corp",  
    "president": {  
        "name": "Dr. Smith",  
        "department": "Leadership"  
    }  
}
```

Creating Nested Dictionaries

- Let's see how we can create an nested Dictionary

```
nested_dict = {  
    "person1": {"name": "Alice", "age": 25, "city": "London"},  
    "person2": {"name": "Bob", "age": 30, "city": "New York"}  
}
```

What's happening here:

- "person1" and "person2" are keys → **outer dictionary**.
- Each one holds its own inner dictionary with details like name, age, and city.

Another Example

```
employees = {  
    "emp1": {"name": "John", "position": "Manager", "age": 35},  
    "emp2": {"name": "Sara", "position": "Developer", "age": 28}  
}  
  
print(employees)
```

Inner Dictionaries

Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/Users/HP/Desktop/tt/test.py  
{'emp1': {'name': 'John', 'position': 'Manager', 'age': 35}, 'emp2': {'name': 'Sara', 'position': 'Developer', 'age': 28}}  
PS C:\Users\HP\Desktop\tt>
```

Accessing Nested Elements

- To get a value from a nested dictionary, use **two keys** —

Syntax

```
nested_dict["outer_key"]["inner_key"]
```

Key for inner Dictionary

Key of Outer Dictionary

Basic Example

```
employees = {  
    "emp1": {"name": "John", "position": "Manager", "age": 35},  
    "emp2": {"name": "Sara", "position": "Developer", "age": 28}  
}
```

```
emp1_name = employees["emp1"]["name"]  
print(emp1_name) # Output: John
```

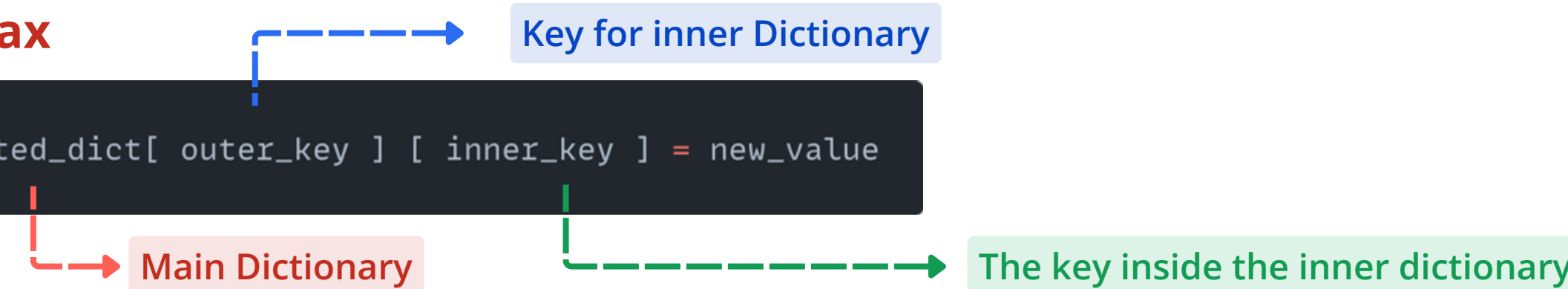
```
emp2_position = employees["emp2"]["position"]  
print(emp2_position) # Output: Developer
```

Modifying Nested Dictionaries

- We can **change, add, or remove** values inside nested dictionaries — just like with **normal dictionaries**.

Syntax

```
nested_dict[ outer_key ] [ inner_key ] = new_value
```

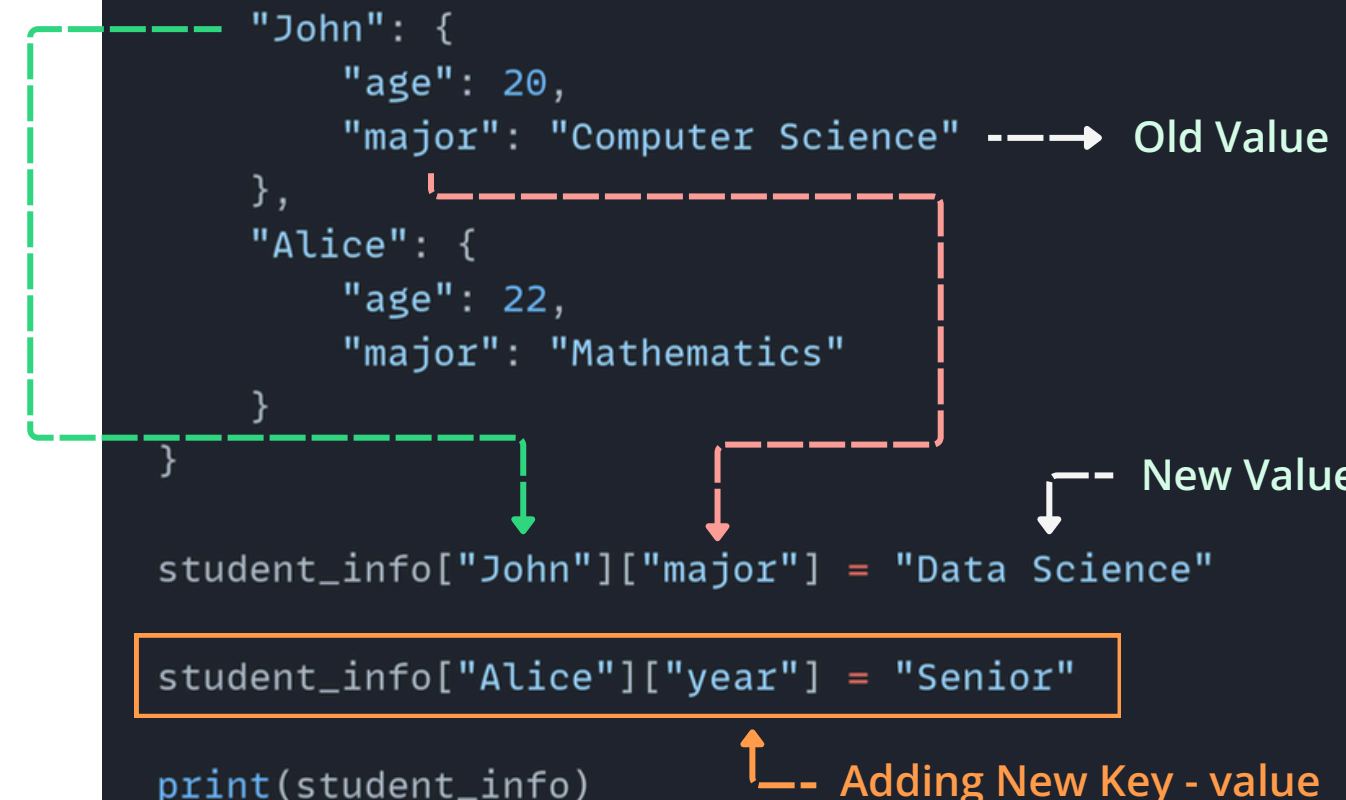


Main Dictionary

Key for inner Dictionary

The key inside the inner dictionary

```
student_info = {  
    "John": {  
        "age": 20,  
        "major": "Computer Science" -- Old Value  
    },  
    "Alice": {  
        "age": 22,  
        "major": "Mathematics"  
    }  
}  
  
student_info["John"]["major"] = "Data Science"  
student_info["Alice"]["year"] = "Senior"  
print(student_info)
```



Old Value

New Value

Adding New Key - value

Output

```
{  
    "John": {"age": 20, "major": "Data Science"},  
    "Alice": {"age": 22, "major": "Mathematics", "year": "Senior"}  
}
```

Summary

Task	What It Does	Example
• Create nested dict	→ Dictionary inside another dictionary	→ <code>{"person1": {"name": "Alice"}}</code>
• Access nested value	→ Gets inner value	→ <code>data["person1"]["name"]</code>
• Update inner value	→ Changes the value	→ <code>data["person1"]["age"] = 26</code>
• Add new inner key	→ Adds a new key-value to inner dict	→ <code>data["person2"]["city"] = "Paris"</code>

Concept Practice Task

Creating a Simple Nested Dictionary

```
student = {
    "name": "John",
    "details": {
        "age": 21,
        "grade": "A" }
}

print(student)
# Output: {'name': 'John', 'details': {'age': 21, 'grade': 'A'}}
```

Modifying a Nested Value

```
student = {
    "name": "John",
    "details": {
        "age": 21,
        "grade": "A" }
}

student["details"]["grade"] = "A+"

print(student)
# Output: {'name': 'John', 'details': {'age': 21, 'grade': 'A+'}}
```

Creating a More Complex Nested Dictionary

```
employee = {
    "name": "Emma",
    "position": "Manager",
    "projects": {
        "current": ["Project A", "Project B"],
        "completed": ["Project X"] }
}

print(employee)
# Output:
# {'name': 'Emma', 'position': 'Manager',
# 'projects': {'current': ['Project A', 'Project B'],
# 'completed': ['Project X']}}
```

Concept Practice Task

Accessing a Nested Value

```
student = {  
    "name": "John",  
    "details": {  
        "age": 21,  
        "grade": "A" }  
}  
age = student["details"]["age"]  
  
print(age)  
# Output: 21
```

Accessing a Nested Value in a Classroom Dictionary

```
classroom = {  
    "class_name": "Physics",  
    "students": {  
        "student1": {"name": "Alice", "age": 20},  
        "student2": {"name": "Bob", "age": 22} }  
}  
  
student_name = classroom["students"]["student2"]["name"]  
  
print(student_name)  
# Output: Bob
```

Concept Practice Task

GamutX

Adding a New Key-Value Pair to a Nested Dictionary

```
employee = {
    "name": "Emma",
    "position": "Manager",
    "projects": {
        "current": ["Project A", "Project B"],
        "completed": ["Project X"] }
}
```

```
employee["projects"]["current"].append("Project C")
```

```
print(employee)
```

```
# Output:
# {'name': 'Emma',
#  'position': 'Manager',
#  'projects': {'current': ['Project A', 'Project B', 'Project C'],
#               'completed': ['Project X']}}
```

Creating a Nested Dictionary for a Classroom

```
classroom = {
    "class_name": "Physics",
    "students": {
        "student1": {"name": "Alice", "age": 20},
        "student2": {"name": "Bob", "age": 22} }
}
```

```
print(classroom)
```

```
# Output:
# {'class_name': 'Physics',
#  'students': {'student1': {'name': 'Alice', 'age': 20},
#               'student2': {'name': 'Bob', 'age': 22}}}
```

Concept Practice Task

Modifying a Nested Value in a Classroom Dictionary

```
classroom = {  
    "class_name": "Physics",  
    "students": {  
        "student1": {"name": "Alice", "age": 20},  
        "student2": {"name": "Bob", "age": 22} }  
}  
  
classroom["students"]["student1"]["age"] = 21  
  
print(classroom)  
# Output:  
# {'class_name': 'Physics',  
# 'students': {'student1': {'name': 'Alice', 'age': 21},  
# 'student2': {'name': 'Bob', 'age': 22}}}
```

Dictionary Methods

- Dictionary Methods
- `get()`
- `keys()`
- `values()`
- `items()`
- `update()`
- `pop()`
- `clear()`

Dictionary Methods

- Python provides some useful **built-in methods to work with dictionaries**.
- These methods help us **access, update, or modify data easily**.

List of Common Dictionary Methods

get()

keys()

values()

items()

update()

pop()

clear()

get()

- Returns the **value for a given key**.
- If the **key does not exist**, it returns **None** or a **custom default value** if provided.
- This Method is safer than using `person["age"]` directly, which gives an error if the key doesn't exist.

Basic Usage

```
person = {"name": "Alice", "age": 25}

age = person.get("age")
city = person.get("city")

print(age)    # Output: 25
print(city)   # Output: None
```

Using get() on an Empty Dictionary

```
empty_dict = {}

result = empty_dict.get("key", "Not Found")

print(result)
# Output: Not Found
```

Accessing a Key That Exists

```
fruits = { "apple": 10, "banana": 20, "orange": 15}

banana_count = fruits.get("banana")

print(banana_count)
# Output: 20
```

Concept Practice Task

GamutX

Accessing Nested Dictionary Values

```
student = {  
    "name": "John",  
    "grades": {  
        "math": 90,  
        "science": 85 }  
}  
  
math_grade = student.get("grades").get("math")  
  
print(math_grade)  
# Output: 90
```

Accessing a Non-Existing Key with Default Value

```
car = {"brand": "Toyota", "model": "Camry", "year": 2021 }  
  
color = car.get("color", "Not Found")  
  
print(color)  
# Output: Not Found
```

keys()

- The **keys()** method in Python dictionaries **allows you to access all the keys present in a dictionary**
- **Returns a view object** containing all the keys in the dictionary.
- Useful when we want to loop through just the keys.

Basic Usage

```
person = {"name": "Alice", "age": 25, "city": "London"}

keys = person.keys()
print(keys)
# Output: dict_keys(['name', 'age', 'city'])
```

Directly Displaying Keys

```
car = { "brand": "Toyota", "model": "Camry", "year": 2021 }

print(car.keys())
# Output: dict_keys(['brand', 'model', 'year'])
```

Assigning keys() to a Variable

```
fruits = { "apple": 10, "banana": 20, "orange": 15 }

fruit_keys = fruits.keys()

print(fruit_keys)
# Output: dict_keys(['apple', 'banana', 'orange'])
```

Accessing Keys in a Nested Dictionary

```
student = {"name": "John", "grades": { "math": 90, "science": 85 }}

grades_keys = student["grades"].keys()

print(grades_keys)
# Output: dict_keys(['math', 'science'])
```

Concept Practice Task

Using keys() with an Empty Dictionary

```
empty_dict = {}  
  
empty_keys = empty_dict.keys()  
  
print(empty_keys)  
  
# Output: dict_keys([])
```

values()

- Returns a view object containing all the values in the dictionary.
- Helps when we **only care about the data**, not the keys.

Basic Usage

```
person = {"name": "Alice", "age": 25, "city": "London"}

values = person.values()
print(values)
# Output: dict_values(['Alice', 25, 'London'])
```

Directly Displaying Values

```
car = { "brand": "Toyota", "model": "Camry", "year": 2021 }

print(car.values())
# Output: dict_values(['Toyota', 'Camry', 2021])
```

Assigning values() to a Variable

```
fruits = { "apple": 10, "banana": 20, "orange": 15 }

fruit_values = fruits.values()

print(fruit_values)
# Output: dict_values([10, 20, 15])
```

Accessing Values in a Nested Dictionary

```
student = { "name": "John", "grades": {"math": 90, "science": 85}}

grades_values = student["grades"].values()

print(grades_values)
# Output: dict_values([90, 85])
```

Concept Practice Task

Using values() with an Empty Dictionary

```
empty_dict = {}  
  
empty_values = empty_dict.values()  
  
print(empty_values)  
# Output: dict_values([])
```

items()

- Returns a view object containing key-value pairs as tuples.
- Commonly used when **looping through both keys and values**.

Basic Usage

```
person = {"name": "Alice", "age": 25, "city": "London"}

items = person.items()
print(items)
# dict_items([('name', 'Alice'), ('age', 25), ('city', 'London')])
```

Assigning items() to a Variable

```
fruits = { "apple": 10, "banana": 20, "orange": 15}

fruit_items = fruits.items()

print(fruit_items)
# Output: dict_items([('apple', 10), ('banana', 20), ('orange', 15)])
```

Viewing Items in a Nested Dictionary

```
student = { "name": "John", "grades": {"math": 90, "science": 85}}

grades_items = student["grades"].items()

print(grades_items)
# Output: dict_items([('math', 90), ('science', 85)])
```

Displaying Items Directly

```
settings = { "volume": 70, "brightness": 50, "language": "English" }

print(settings.items())

# dict_items([('volume', 70), ('brightness', 50), ('language', 'English')])
```

update()

- Updates the **dictionary with values from another dictionary**.
- It will **overwrite existing keys and add new ones**.

Basic Usage

```
person = {"name": "Alice", "age": 25}

person.update({"age": 26, "city": "London"})
print(person)
# Output: {'name': 'Alice', 'age': 26, 'city': 'London'}
```

Merging Two Dictionaries

```
dict1 = {"name": "John", "age": 30 }

dict2 = { "city": "Los Angeles", "country": "USA" }
dict1.update(dict2)

print(dict1)
# {'name': 'John', 'age': 30, 'city': 'Los Angeles', 'country': 'USA'}
```

Updating Multiple Keys at Once

```
student = {
    "name": "Emma",
    "grade": "B"
}

student.update({"grade": "A", "age": 22})

print(student)
# Output: {'name': 'Emma', 'grade': 'A', 'age': 22}
```

Adding Key-Value Pairs

```
person = {
    "name": "Alice",
    "age": 25
}

person.update({"city": "New York"})

print(person)
# Output: {'name': 'Alice', 'age': 25, 'city': 'New York'}
```

Concept Practice Task

GamutX

Updating Existing Key Values

```
car = {  
    "brand": "Toyota",  
    "model": "Camry",  
    "year": 2021  
}
```

```
car.update({"year": 2022})
```

```
print(car)
```

```
# Output: {'brand': 'Toyota', 'model': 'Camry', 'year': 2022}
```

pop()

- Removes the key from the dictionary and returns its value.
- **Raises a KeyError if the key doesn't exist** (unless a default value is provided).
- Use this when **we need to delete and also use the value**.

Basic Usage

```
person = {"name": "Alice", "age": 25, "city": "London"}

age = person.pop("age")
print(age)           # 25
print(person)        # {'name': 'Alice', 'city': 'London'}
```

Using pop() with a Non-Existing Key

```
car = {"brand": "Toyota", "model": "Camry", "year": 2021 }
removed_value = car.pop("color", "Key not found")

print(car)
# {'brand': 'Toyota', 'model': 'Camry', 'year': 2021}

print("Removed value:", removed_value)
# Output: Removed value: Key not found
```

Removing a Nested Dictionary Key-Value Pair

```
student = { "name": "John", "details": { "grade": "A", "age": 21 } }

removed_value = student["details"].pop("age")

print(student)
# Output: {'name': 'John', 'details': {'grade': 'A'}}

print("Removed value:", removed_value)
# Output: Removed value: 21
```

clear()

- The **clear()** method removes everything from the dictionary, resulting in an empty dictionary {}.

Basic Usage

```
person = {"name": "Alice", "age": 25, "city": "London"}

person.clear()

print(person) # Output: {}
```

Clearing a Nested Dictionary

```
employee = { "name": "Emma", "position": "Manager",
             "projects": {
                 "current": ["Project A", "Project B"],
                 "completed": ["Project X"]
             }
}

employee.clear()

print(employee)
# Output: {}
```

Checking the Dictionary Before and After Clearing

```
settings = {
    "volume": 70,
    "brightness": 50,
    "language": "English"
}

print("Before clear():", settings)
# Before clear(): {'volume': 70, 'brightness': 50, 'language': 'English'}

settings.clear()

print("After clear():", settings)
# Output: After clear(): {}
```

Summary Table

GamutX

Method	Description	Example Usage
• get()	→ Gets the value of a key (returns None if missing)	→ person.get("name")
• keys()	→ Returns all keys as a view object	→ person.keys()
• values()	→ Returns all values as a view object	→ person.values()
• items()	→ Returns key-value pairs as tuples	→ person.items()
• update()	→ Updates or adds multiple key-value pairs	→ person.update({"city": "London"})
• pop()	→ Removes a key and returns its value	→ person.pop("age")
• clear()	→ Removes everything from the dictionary	→ person.clear()

Common Errors with Dictionary

- KeyError
- TypeError
- ValueError

KeyError

- This error happens when you **try to access a key that doesn't exist in the dictionary**.

Causes

```
my_dict = {"name": "Alice", "age": 25}

print(my_dict["city"])
```

```
⊗ PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/U
Traceback (most recent call last):
  File "c:\Users\HP\Desktop\tt\test.py", line 2, in <module>
    print(my dict["city"])
  KeyError: 'city'
○ PS C:\Users\HP\Desktop\tt>
```

Prevention

```
print(my_dict.get("city"))           # Output: None
print(my_dict.get("city", "Not Found")) # Output: Not Found
```

Use the **get() method instead**. It won't give an error if the key is missing.

TypeError

- This error happens when you **use a list, dictionary, or any other unhashable (changeable) type as a key.**
- **Only immutable types like strings, numbers, and tuples** can be used as dictionary keys.

Causes

```
my_dict = {"name": "Alice", "age": 25}

my_dict = {[1, 2]: "value"}
```

```
⊗ PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/Users/HP/
Traceback (most recent call last):
  File "c:\Users\HP\Desktop\tt\test.py", line 3, in <module>
    my_dict = {[1, 2]: "value"}
TypeError: unhashable type: 'list'
○ PS C:\Users\HP\Desktop\tt>
```

Prevention

```
my_dict = {1: "value"}           # Integer key
my_dict = {("a", "b"): "data"}  # Tuple key
```

Use only strings, integers, or tuples as keys.

ValueError

- This **error happens when you try to update a dictionary using the wrong format.**
- The update() method **expects key-value pairs.**

Causes

```
my_dict = {"name": "Alice", "age": 25}

my_dict.update(["incorrect_format"])
```

Prevention

```
my_dict.update([("city", "London")])

print(my_dict)
# {'name': 'Alice', 'age': 25, 'city': 'London'}
```

Use a proper format like another dictionary or a list of tuples.

```
PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/Users/HP/Desktop/tt/test.py
Traceback (most recent call last):
  File "c:\Users\HP\Desktop\tt\test.py", line 2, in <module>
    my_dict.update(["incorrect_format"])
ValueError: dictionary update sequence element #0 has length 16; 2 is required
PS C:\Users\HP\Desktop\tt>
```

Points to Remember:

- Key:
 - A key is the identifier or label used to access the corresponding value.
 - Keys must be unique within a dictionary.
 - If a key is repeated, the last value assigned to that key will overwrite the previous one.
 - Keys must be of an immutable type (e.g., strings, numbers, or tuples).
 - We cannot use a list or another dictionary as a key because they are mutable.
- Value:
 - A value is the data associated with a key.
 - Values can be of any data type, including strings, numbers, lists, or even other dictionaries. They don't need to be unique, and they can be mutable.
- Key-Value Pair:
 - A key-value pair is written in the format: key: value.
 - Multiple key-value pairs are separated by commas.

Thank You

